Web Engineering & Development (SWE 363)

Web Security

In today's lecture:

- Third-party web APIs
- Token-based user authentication (JWT)
- Password hashing

Reference:

- Zybook: 8.1-8.3

Activity: Think-Pair-Share (2 min)

You log in to Blackboard once, then open many pages without typing your password again.

1. How could the server know it is still you on every request?
2. Should the server store your actual password? What could it store instead?
3. A weather widget on the page shows today's forecast. Who produced that data?

8.1 Third-Party Web APIs

Third-Party Web APIs

> A **third-party web API** is a web API hosted by another person or organization that any web client may use

- Why: weather data, maps, and more, without building them
- Usually needs an **API key** that identifies who is using it

```
http://api.openweathermap.org/data/2.5/weather?zip=90210&units=imperial&appid=APIKEY
```

Calling a Third-Party API with fetch()

> The web server sends the request with **fetch()** and gets **JSON** back

```
app.get("/weather", function(req, res) {
  const zip = 90210;
  const params = new URLSearchParams({
    zip: zip, units: "imperial", appid: "APIKEY"});
  fetch("http://api.openweathermap.org/data/2.5/weather?" + params)
    .then(response => response.json())       // JSON → JavaScript objects
    .then(data => res.render("weather", { data: data, zip: zip }))
    .catch(error => console.log(error))
});
```

- `URLSearchParams` builds the query string; `fetch()` needs **Node 18** or above

Activity: Predict

```
const params = new URLSearchParams({
  zip: 10001, units: "metric", appid: "KEY"});
fetch("http://api.openweathermap.org/data/2.5/weather?"
  + params)
  .then(response => response.json())
  .then(data => {
    console.log(data.main.temp);
  });
```

Part of the JSON response:

```
{ "main": { "temp": 21.5 },
  "wind": { "speed": 4.1 } }
```

1. When does the first then() run?
2. When does the second then() run?
3. Wind speed: data.main.temp, data.wind.speed, or data.main.speed?
4. Celsius: standard, metric, or imperial?
5. What query string does params create?

8.2 Token-Based User Authentication

Token-Based Authentication

> **Token-based authentication:** the client uses a **signed token** to "prove" to the server that it has successfully authenticated

- Why: the user logs in **once**, not on every request

```
1. POST /api/auth   username + password  → server sends a token
2. Client stores the token
3. GET /api/status  X-Auth: token        → server validates it, sends data
```

JSON Web Tokens (JWT)

> A **JWT** is a string with three parts, **header.payload.signature**, signed with a **secret key** known only by the server

- Why: if the client changes the payload, the signature no longer matches

```
header     eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9            {"typ":"JWT","alg":"HS256"}
payload    eyJ1c2VybmFtZSI6ImJvYiJ9                        {"username":"bob"}
signature  Mm0fNOZMBFOrFu99NlnHdz3jkp5IE_BQCNOz4sh1epQ     secret key "supersecret"
```

jwt-simple: encode() and decode()

> The **jwt-simple** module creates and validates JWTs: `npm install jwt-simple`

```
const jwt = require("jwt-simple");
const secret = "supersecret";

// Create a JWT (after a successful login)
const token = jwt.encode({ username: "bsmith" }, secret);

// Decode a JWT (on later requests)
const decoded = jwt.decode(token, secret);
console.log(decoded.username);   // bsmith
```

- A changed or fake token makes `jwt.decode()` fail

The X-Auth Header

> The client sends the token in the **X-Auth** header; the server validates it

- Login route: correct password → `jwt.encode()` → token; else **401**

```
router.get("/status", function(req, res) {
  const token = req.headers["x-auth"];
  try {
    const decoded = jwt.decode(token, secret);
    res.json({status: "Enjoying a beautiful day!"});
  }
  catch (ex) {
    res.status(401).json({ error: "Invalid JWT" });
  }
});
```

Activity: What happens if... (status code + message)

```
POST /api/auth    not bsmith / pass          → 401 { error: "Bad username/password" }
GET  /api/status  no X-Auth header           → 401 { error: "Missing X-Auth header" }
GET  /api/status  jwt.decode(X-Auth) fails → 401 { error: "Invalid JWT" }
Success: /api/auth → { token: ... }, /api/status → { status: ... }
```

1. POST /api/auth with username=bmsith&password=opensesame
2. GET /api/status with no X-Auth header
3. GET /api/status with X-Auth: bsmith
4. One character of a valid token is changed, then GET /api/status

8.3 Password Hashing

Password Hashing

> A **cryptographic hash function** converts text of any length into a fixed-length **hash**; the hash can't be directly converted back

- Why: **never store passwords**; store hashes, so a stolen database doesn't reveal them

```
Register:  opensesame → hash → BC029G8A38   stored in the database
Login:     OpenSesame → hash → 75EE39FA90   ≠ stored → denied
Login:     opensesame → hash → BC029G8A38   = stored → correct password
```

Salt

> A **salt** is a random string combined with a password so two identical passwords produce **different hashes**

- Why: cracking salted hashes is much harder

```
opensesame + salt 33M.4Zn7R7k3jHOISHxCe. → yUqI4vl9mv4/oeiLuhHDQZcfySVg6wC
opensesame + salt ByrTcHizkol25k1WOnlV2e → gFI3DdRzo9.xrjal/IyiaKd8X5diFpi
```

bcryptjs: hashSync() and compareSync()

> **bcryptjs** (`npm install bcryptjs`) hashes passwords with a random salt

```
const bcrypt = require("bcryptjs");

// Register: store the hash, not the password
const hash = bcrypt.hashSync("opensesame", 10);   // 10 = cost factor

// Login: compare the submitted password with the stored hash
if (bcrypt.compareSync("opensesame", hash)) {
  console.log("Same!");
}
```

- Same password → a different hash each time, so don't compare hashes with `===`

Activity: Predict

```
const hash = bcrypt.hashSync("qwerty", 10);
if (bcrypt.compareSync("Qwerty", hash)) {
  console.log("Same!");
}
else {
  console.log("Not the same");
}
```

1. What does the code output?
2. What if hashSync("qwerty", 10) runs twice: are the two hashes equal?
3. What if the cost factor changes from 10 to 12: does hashSync() take less or more time?
4. Can a website that stores only password hashes recover a user's lost password?

Activity: Exit ticket: find the 4 bugs

users is an object: username → saved password data.

```
router.post("/user", function(req, res) {
  users[req.body.username] = req.body.password;
  res.json({ message: "User added" });
});
router.post("/auth", function(req, res) {
  const hash = bcrypt.hashSync(req.body.password, 10);
  if (hash === users[req.body.username]) {
    res.json({ token: jwt.encode({ username: req.body.username }) });
  } else { res.status(200).json({ error: "Bad username/password" }); }
});
```

Key Takeaways

- A **third-party web API** is called from the server with `fetch()`; usually needs an **API key**
- **Token-based auth:** log in once, then send a **signed token** with each request
- **JWT** = header.payload.signature, signed with the server's **secret key**
- jwt-simple: `jwt.encode()` / `jwt.decode()`; token in the **X-Auth** header
- **Never store passwords:** store hashes; **salt** makes equal passwords hash differently
- bcryptjs: `hashSync()` to register, `compareSync()` to log in

30m

Demo

8.1 Third-Party APIs and Authentication

Build a secure weather API with:

- Password hashing
- Token-based authentication
- A third-party weather API

Next Class

- Course review
- Final project work